

ALGORITMO E ESTRUTURA DE DADOS

Aula: Matrizes em Linguagem C

Centro Universitário CEUNI - FAMETRO

3º Período - Engenharia da Computação 2026.1

Passo a Passo com Exemplos Práticos e Exercícios Resolvidos

Professor Alexsander Farias

🎯 Objetivos da Aula

- Compreender o conceito de matrizes bidimensionais
- Declarar, inicializar e acessar elementos de matrizes em C
- Utilizar laços aninhados para percorrer matrizes
- Implementar operações clássicas: soma, transposição e multiplicação
- Resolver problemas práticos aplicando matrizes

1. O QUE É UMA MATRIZ?

Uma **matriz** (ou array bidimensional) é uma estrutura de dados que organiza elementos em **linhas** e **colunas**, formando uma grade ou tabela. É a extensão natural dos vetores (arrays unidimensionais) para duas dimensões.

i Conceito fundamental

Em C, uma matriz é um array de arrays. Cada linha é um vetor, e o conjunto de todas as linhas forma a matriz.

Acesso ao elemento: `matriz[linha][coluna]` → O índice começa em 0!

1.1 Visualização de uma Matriz 3×4

Exemplo de uma matriz com 3 linhas (i) e 4 colunas (j):

	j=0	j=1	j=2	j=3
i=0	10	20	30	40
i=1	50	60	70	80
i=2	90	100	110	120

O elemento destacado (laranja) é `matriz[1][2] = 70`. Lembre-se: índices começam em 0.

Termo	Em C	Exemplo
Linha (m)	Primeira dimensão	<code>int m[3][4]</code> → 3 linhas
Coluna (n)	Segunda dimensão	<code>int m[3][4]</code> → 4 colunas
Elemento	<code>m[i][j]</code>	<code>m[0][0]</code> = primeiro elemento
Tamanho total	<code>m × n</code> elementos	<code>3×4 = 12</code> elementos

2. DECLARAÇÃO E INICIALIZAÇÃO

2.1 Declaração de Matrizes

Sintaxe geral: `tipo nome[linhas][colunas];`

```
/* Formas de declarar matrizes em C */

int    mat[3][4];           // Matriz 3x4 de inteiros (não inicializada)
float  notas[5][10];        // Matriz 5x10 de ponto flutuante
char   tabuleiro[8][8];      // Tabuleiro de xadrez (8x8 chars)
double tabela[100][100];     // Matriz 100x100 de doubles
```

2.2 Inicialização na Declaração

```
/* Inicialização com valores explícitos */
int mat[3][4] = {
    {10, 20, 30, 40}, // Linha 0
    {50, 60, 70, 80}, // Linha 1
    {90, 100, 110, 120} // Linha 2
};

/* Inicialização linear (C preenche linha por linha) */
int mat2[2][3] = {1, 2, 3, 4, 5, 6};
/* Equivale a: { {1,2,3}, {4,5,6} } */

/* Inicialização parcial (demais = 0) */
int zeros[3][3] = {0}; // Todos os elementos = 0
```

⚠ Dica importante

Ao declarar uma matriz sem inicializar, ela pode conter lixo de memória (valores aleatórios). Sempre inicie ou preencha a matriz antes de usá-la!

2.3 Como C Armazena a Matriz na Memória

C armazena matrizes em **ordem de linha principal (row-major order)**: os elementos de cada linha ficam contíguos na memória.

`int mat[2][3] → Na memória:`

`[ mat[0][0] | mat[0][1] | mat[0][2] | mat[1][0] | mat[1][1] | mat[1][2] ]`
*Endereço: &mat[i][j] = &mat[0][0] + (i * colunas + j) * sizeof(tipo)*

3. PERCORRENDO MATRIZES COM LAÇOS ANINHADOS

O padrão para percorrer uma matriz usa **dois laços for aninhados**: o externo para linhas e o interno para colunas.

3.1 Leitura e Impressão - Exemplo Completo

```
#include <stdio.h>

#define LINHAS 3
#define COLUNAS 4

int main() {
    int mat[LINHAS][COLUNAS];
    int i, j;

    /* ----- LEITURA ----- */
    printf("Digite os %d valores da matriz:\n", LINHAS * COLUNAS);
    for (i = 0; i < LINHAS; i++) {
        for (j = 0; j < COLUNAS; j++) {
            printf("mat[%d][%d]: ", i, j);
            scanf("%d", &mat[i][j]);
        }
    }

    /* ----- IMPRESSÃO ----- */
    printf("\nMatriz lida:\n");
    for (i = 0; i < LINHAS; i++) {
        for (j = 0; j < COLUNAS; j++) {
            printf("%6d", mat[i][j]);    // %6d: 6 casas para alinhar
        }
        printf("\n");    // Quebra de linha ao final de cada linha
    }

    return 0;
}
```

► Saída do programa:

```
Digite os 12 valores da matriz:
mat[0][0]: 10  mat[0][1]: 20  mat[0][2]: 30  mat[0][3]: 40
mat[1][0]: 50  mat[1][1]: 60  mat[1][2]: 70  mat[1][3]: 80
mat[2][0]: 90  mat[2][1]: 100 mat[2][2]: 110 mat[2][3]: 120
```

```
Matriz lida:
  10    20    30    40
  50    60    70    80
  90   100   110   120
```

4. OPERAÇÕES CLÁSSICAS COM MATRIZES

4.1 Soma de Duas Matrizes

Somar duas matrizes A e B de mesma dimensão resulta em uma matriz C onde cada elemento é $C[i][j] = A[i][j] + B[i][j]$.

```
#include <stdio.h>
#define N 3

void imprimir(int m[N][N]) {
    int i, j;
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++)
            printf("%5d", m[i][j]);
        printf("\n");
    }
}

int main() {
    int A[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int B[N][N] = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
    int C[N][N];
    int i, j;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            C[i][j] = A[i][j] + B[i][j];

    printf("Matriz A:\n");  imprimir(A);
    printf("Matriz B:\n");  imprimir(B);
    printf("A + B:\n");     imprimir(C);
    return 0;
}
```

► Saída do programa:

Matriz A:			Matriz B:			A + B:		
1	2	3	9	8	7	10	10	10
4	5	6	6	5	4	10	10	10
7	8	9	3	2	1	10	10	10

4.2 Transposição de Matrizes

A **transposta** de uma matriz $m \times n$ é uma matriz $n \times m$ onde $T[j][i] = M[i][j]$.

```

#include <stdio.h>
#define LINHAS 3
#define COLUNAS 4

int main() {
    int M[LINHAS][COLUNAS] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    int T[COLUNAS][LINHAS]; // Dimensões invertidas!
    int i, j;

    /* Gerar a transposta */
    for (i = 0; i < LINHAS; i++)
        for (j = 0; j < COLUNAS; j++)
            T[j][i] = M[i][j];

    /* Imprimir M */
    printf("Matriz M (3x4):\n");
    for (i = 0; i < LINHAS; i++) {
        for (j = 0; j < COLUNAS; j++) printf("%4d", M[i][j]);
        printf("\n");
    }

    /* Imprimir T */
    printf("\nTransposta T (4x3):\n");
    for (i = 0; i < COLUNAS; i++) {
        for (j = 0; j < LINHAS; j++) printf("%4d", T[i][j]);
        printf("\n");
    }
    return 0;
}

```

► Saída do programa:

Matriz M (3x4):	Transposta T (4x3):
1 2 3 4	1 5 9
5 6 7 8	2 6 10
9 10 11 12	3 7 11
	4 8 12

4.3 Multiplicação de Matrizes

Multiplica-se A ($m \times n$) por B ($n \times p$), resultando em C ($m \times p$). Cada elemento: $C[i][j] = \sum A[i][k] * B[k][j]$.

i Regra de ouro da multiplicação

O número de COLUNAS de A deve ser igual ao número de LINHAS de B.

$A(m \times n) \times B(n \times p) = C(m \times p)$

```

#include <stdio.h>
#define M 2 // linhas de A

```

```
#define N 3    // colunas de A = linhas de B
#define P 2    // colunas de B

int main() {
    int A[M][N] = {{1, 2, 3}, {4, 5, 6}};
    int B[N][P] = {{7, 8}, {9, 10}, {11, 12}};
    int C[M][P] = {0};    // Inicializa com zeros!
    int i, j, k;

    /* Multiplicação: 3 laços aninhados */
    for (i = 0; i < M; i++)
        for (j = 0; j < P; j++)
            for (k = 0; k < N; k++)
                C[i][j] += A[i][k] * B[k][j];

    printf("Resultado C = A x B (2x2):\n");
    for (i = 0; i < M; i++) {
        for (j = 0; j < P; j++) printf("%6d", C[i][j]);
        printf("\n");
    }
    return 0;
}
```

► Saída do programa:

```
Resultado C = A x B (2x2):
    58    64
   139   154
```

```
C[0][0] = 1*7 + 2*9 + 3*11 = 7 + 18 + 33 = 58
C[0][1] = 1*8 + 2*10 + 3*12 = 8 + 20 + 36 = 64
C[1][0] = 4*7 + 5*9 + 6*11 = 28 + 45 + 66 = 139
C[1][1] = 4*8 + 5*10 + 6*12 = 32 + 50 + 72 = 154
```

5. PASSAGEM DE MATRIZES PARA FUNÇÕES

Em C, ao passar uma matriz para uma função, deve-se especificar o **número de colunas** na assinatura da função (o número de linhas pode ser omitido ou passado como parâmetro).

```
#include <stdio.h>
#define COLS 4

/* Função recebe matriz com número fixo de colunas */
void preencherZeros(int mat[][COLS], int linhas) {
    int i, j;
    for (i = 0; i < linhas; i++)
        for (j = 0; j < COLS; j++)
            mat[i][j] = 0;
}

/* Função calcula a soma de todos os elementos */
int somaTotal(int mat[][COLS], int linhas) {
    int i, j, soma = 0;
    for (i = 0; i < linhas; i++)
        for (j = 0; j < COLS; j++)
            soma += mat[i][j];
    return soma;
}
```

```
    for (i = 0; i < linhas; i++)
        for (j = 0; j < COLS; j++)
            soma += mat[i][j];
    return soma;
}

int main() {
    int tabela[3][COLS] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
    printf("Soma: %d\n", somaTotal(tabela, 3));

    preencherZeros(tabela, 3);
    printf("Soma apos zerar: %d\n", somaTotal(tabela, 3));
    return 0;
}
```

► Saída do programa:

Soma: 78
Soma apos zerar: 0

i Por que precisa informar as colunas?

C precisa saber o tamanho das colunas para calcular o endereço correto de `mat[i][j]` na memória.

Fórmula: endereço = base + (i * COLS + j) * sizeof(tipo)

Sem saber COLS, o compilador não consegue calcular o offset correto.

6. APLICAÇÃO PRÁTICA: SISTEMA DE NOTAS

Aplicação completa: sistema de cadastro de notas de alunos usando matrizes.

```
#include <stdio.h>
#define ALUNOS 4
#define PROVAS 3

int main() {
    float notas[ALUNOS][PROVAS];
    float media[ALUNOS];
    int i, j;

    /* Leitura das notas */
    printf("=== SISTEMA DE NOTAS ===\n\n");
    for (i = 0; i < ALUNOS; i++) {
        printf("Aluno %d - Digite %d notas: ", i+1, PROVAS);
        for (j = 0; j < PROVAS; j++)
            scanf("%f", &notas[i][j]);
    }

    /* Cálculo das médias */
    for (i = 0; i < ALUNOS; i++) {
        float soma = 0;
        for (j = 0; j < PROVAS; j++)
            soma += notas[i][j];
    }
}
```

```

        media[i] = soma / PROVAS;
    }

    /* Relatório */
    printf("\n%-10s %8s %8s %8s %10s %12s\n",
           "Aluno", "P1", "P2", "P3", "Media", "Situacao");
    printf("-----\n");

    for (i = 0; i < ALUNOS; i++) {
        printf("Aluno %-4d ", i+1);
        for (j = 0; j < PROVAS; j++)
            printf("%8.1f", notas[i][j]);
        printf("%10.1f %12s\n",
               media[i],
               media[i] >= 6.0 ? "APROVADO" : "REPROVADO");
    }
    return 0;
}

```

► Saída do programa:

```
=== SISTEMA DE NOTAS ===
```

```

Aluno 1 - Digite 3 notas: 7.5 8.0 9.0
Aluno 2 - Digite 3 notas: 5.0 4.5 6.0
Aluno 3 - Digite 3 notas: 9.0 9.5 10.0
Aluno 4 - Digite 3 notas: 3.0 4.0 2.0

```

Aluno	P1	P2	P3	Media	Situacao
Aluno 1	7.5	8.0	9.0	8.2	APROVADO
Aluno 2	5.0	4.5	6.0	5.2	REPROVADO
Aluno 3	9.0	9.5	10.0	9.5	APROVADO
Aluno 4	3.0	4.0	2.0	3.0	REPROVADO

7. EXERCÍCIOS RESOLVIDOS

A seguir, exercícios com enunciado completo, código-solução e explicação detalhada.

Questão 1

Escreva um programa em C que leia uma matriz 4×4 de inteiros e determine: (a) o maior elemento; (b) a soma da diagonal principal.

Resposta:

Conceito: diagonal principal → elementos onde $i == j$ ($mat[0][0]$, $mat[1][1]$, $mat[2][2]$, $mat[3][3]$)

```

#include <stdio.h>
#define N 4

```



```
int main() {
    int mat[N][N] = {
        {5, 3, 8, 2},
        {1, 9, 4, 7},
        {6, 0, 12, 3},
        {2, 5, 1, 11}
    };
    int i, j;
    int maior = mat[0][0];
    int somaDiag = 0;

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (mat[i][j] > maior)
                maior = mat[i][j];        // (a) Busca o maior
        }
        somaDiag += mat[i][i];            // (b) Diagonal: i == j
    }

    printf("Maior elemento: %d\n", maior);
    printf("Soma da diagonal principal: %d\n", somaDiag);
    return 0;
}
```

► Saída do programa:

Maior elemento: 12
Soma da diagonal principal: 5 + 9 + 12 + 11 = 37

Explicação: Para o maior, verificamos **todos** os elementos (N^2 comparações). Para a diagonal, percorremos apenas os N elementos onde $i == j$, usando somente o laço externo com `mat[i][i]`.

** Questão 2**

Crie um programa que verifique se uma matriz 3×3 é simétrica. Uma matriz é simétrica quando `mat[i][j] == mat[j][i]` para todo i e j .

** Resposta:**

Conceito: matriz simétrica $\rightarrow M = M^T$ (igual à sua transposta). Basta verificar o triângulo superior.

```
#include <stdio.h>
#define N 3

int main() {
    int mat[N][N] = {
        {1, 2, 3},
        {2, 5, 6},
        {3, 6, 9}
    };
    int i, j;
    int simetrica = 1;    // Assume verdadeiro (1)
}
```

```
/* Verifica apenas o triângulo superior (j > i) */
for (i = 0; i < N && simetrica; i++) {
    for (j = i + 1; j < N && simetrica; j++) {
        if (mat[i][j] != mat[j][i])
            simetrica = 0; // Encontrou diferença: falso
    }
}

if (simetrica)
    printf("A matriz E simetrica.\n");
else
    printf("A matriz NAO e simetrica.\n");

return 0;
}
```

► Saída do programa:

A matriz E simétrica.

Verificações: mat[0][1]=2 == mat[1][0]=2 ✓
mat[0][2]=3 == mat[2][0]=3 ✓
mat[1][2]=6 == mat[2][1]=6 ✓

Explicação: Só verificamos o **triângulo superior** (j começa em i+1), evitando comparações desnecessárias e divisão por zero. A variável `simetrica` age como flag booleana: interrompe os laços imediatamente ao encontrar a primeira assimetria.

 Questão 3

Implemente um programa que leia uma matriz 5×5 e conte quantos elementos são: (a) positivos, (b) negativos e (c) zeros.

```
#include <stdio.h>
#define N 5

int main() {
    int mat[N][N] = {
        { 3, -1, 0, 5, -2},
        { 0, 7, -4, 2, 0},
        {-3, 1, 0, -5, 6},
        { 0, 0, 8, -1, 3},
        { 2, -6, 0, 4, 0}
    };
    int i, j;
    int pos = 0, neg = 0, zeros = 0;

    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) {
            if (mat[i][j] > 0) pos++;
            else if (mat[i][j] < 0) neg++;
            else zeros++;
        }
    }
}
```

```
    }  
}  
  
printf("Total de elementos: %d\n", N * N);  
printf("Positivos: %d\n", pos);  
printf("Negativos: %d\n", neg);  
printf("Zeros:      %d\n", zeros);  
printf("Verificacao: %d + %d + %d = %d\n",  
       pos, neg, zeros, pos + neg + zeros);  
return 0;  
}
```

► Saída do programa:

```
Total de elementos: 25  
Positivos: 11  
Negativos: 8  
Zeros: 6  
Verificação: 11 + 8 + 6 = 25
```

Explicação: Um único laço duplo percorre todos os N^2 elementos. O uso de `if / else if / else` garante que cada elemento seja contado exatamente uma vez. A verificação final (soma dos contadores = $N \times N$) valida a lógica.

📄 Questão 4

Escreva um programa que leia duas matrizes A e B (3×3) e calcule o produto $A \times B$, imprimindo o resultado formatado.

✅ Resposta:

*Este exercício aplica diretamente a seção 4.3. $C[i][j]$ = soma de $A[i][k] * B[k][j]$ para k de 0 a N-1.*

```
#include <stdio.h>  
#define N 3  
  
void lerMatriz(int m[N][N], char nome) {  
    int i, j;  
    printf("\nDigite a Matriz %c (%dx%d):\n", nome, N, N);  
    for (i = 0; i < N; i++)  
        for (j = 0; j < N; j++) {  
            printf("  %c[%d][%d]: ", nome, i, j);  
            scanf("%d", &m[i][j]);  
        }  
}  
  
void imprimirMatriz(int m[N][N], char *titulo) {  
    int i, j;  
    printf("\n%s\n", titulo);  
    for (i = 0; i < N; i++) {  
        printf("  [");  
        for (j = 0; j < N; j++)  
            printf("%6d", m[i][j]);  
    }  
}
```

```

        printf("  ]\n");
    }
}

int main() {
    int A[N][N], B[N][N], C[N][N] = {0};
    int i, j, k;

    lerMatriz(A, 'A');
    lerMatriz(B, 'B');

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                C[i][j] += A[i][k] * B[k][j];

    imprimirMatriz(A, "Matriz A:");
    imprimirMatriz(B, "Matriz B:");
    imprimirMatriz(C, "Produto A x B:");
    return 0;
}

```

► Saída do programa:

Exemplo com $A = \{\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8, 9\}\}$ e $B = \{\{9, 8, 7\}, \{6, 5, 4\}, \{3, 2, 1\}\}$:

Matriz A:	Matriz B:	Produto A x B:
[1 2 3]	[9 8 7]	[30 24 18]
[4 5 6]	[6 5 4]	[84 69 54]
[7 8 9]	[3 2 1]	[138 114 90]

** Questão 5**

DESAFIO - Rotação de Matriz 90°: Escreva um programa que rotacione uma matriz $N \times N$ em 90 graus no sentido horário. Dica: o elemento $mat[i][j]$ vai para $rotacionada[j][N-1-i]$.

```

#include <stdio.h>
#define N 4

void imprimir(int m[N][N], char *titulo) {
    int i, j;
    printf("%s\n", titulo);
    for (i = 0; i < N; i++) {
        for (j = 0; j < N; j++) printf("%4d", m[i][j]);
        printf("\n");
    }
    printf("\n");
}

int main() {
    int mat[N][N] = {
        { 1, 2, 3, 4},
        { 5, 6, 7, 8},
        { 9, 10, 11, 12},
        {13, 14, 15, 16}
    };
}

```

```

};
int rot[N][N];
int i, j;

/* Rotação 90° horária: rot[j][N-1-i] = mat[i][j] */
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        rot[j][N-1-i] = mat[i][j];

imprimir(mat, "Original:");
imprimir(rot, "Rotacionada 90 graus:");
return 0;
}

```

► Saída do programa:

Original:	Rotacionada 90 graus:
1 2 3 4	13 9 5 1
5 6 7 8	14 10 6 2
9 10 11 12	15 11 7 3
13 14 15 16	16 12 8 4

Raciocínio: a primeira coluna da original vira a primeira linha da rotacionada, mas em ordem invertida (de baixo para cima → da esquerda para direita).

Explicação: A fórmula $rot[j][N-1-i] = mat[i][j]$ mapeia corretamente cada posição. O índice $N-1-i$ inverte a linha (última linha vai para coluna 0 da rotacionada). Este padrão é muito usado em processamento de imagens e jogos.

8. RESUMO E BOAS PRÁTICAS

Operação	Sintaxe / Dica
Declaração	<code>int mat[LINHAS][COLUNAS];</code>
Acesso	<code>mat[i][j]</code> — índices começam em 0!
Percorrer	for duplo: <code>externo=linhas, interno=colunas</code>
Inicializar zeros	<code>int mat[M][N] = {0};</code>
Diagonal principal	acessar <code>mat[i][i]</code> com laço único
Transposta	<code>T[j][i] = M[i][j]</code>
Multiplicação	<code>C[i][j] += A[i][k] * B[k][j]</code> (3 laços)
Passagem p/ função	<code>void f(int m[][COLUNAS], int linhas)</code>
Tamanho em bytes	<code>sizeof(mat) = LINHAS * COLUNAS * sizeof(tipo)</code>

8.1 Erros Comuns - Evite!

✖ ✖ Índice fora dos limites (Buffer Overflow)

```
int mat[3][3]; mat[3][0] = 5; // ERRADO! Índice máximo é 2
```

C não verifica limites em tempo de execução — causa comportamento indefinido!

⚠ ✖ Esquecer de inicializar

```
int mat[5][5]; printf("%d", mat[0][0]); // Lixo de memória!
```

Sempre inicialize: `int mat[5][5] = {0};` ou preencha com laços.

⚠ ✖ Confundir linhas e colunas na passagem para funções

```
void errado(int mat[3][], int cols) { } // ERRO DE COMPILAÇÃO
```

```
void correto(int mat[][3], int linhas) { } // Correto: colunas fixas
```

- Fim da Aula -